

BiteFixes Enterprise Architecture Manual

Volume 13

Platform Security Architecture (PSA)

Version: 1.0

Classification: Enterprise Architecture

Status: Draft

Table of Contents

13.1 Purpose

13.2 Security Vision

13.3 Security Principles

13.4 Enterprise Security Layers

13.5 Identity & Access Management (IAM)

13.6 Authentication Architecture

13.7 Authorization Architecture

13.8 Multi-Tenant Security

13.9 API Security

13.10 Application Security

13.11 Database Security

13.12 AI Security (Bitey Security Model)

13.13 Data Privacy

13.14 Secrets Management

13.15 Audit & Logging

13.16 Threat Detection

13.17 Backup & Disaster Recovery

13.18 External Integrations Security

13.19 Compliance Strategy

13.20 Security Roadmap

13.1 Purpose

The Platform Security Architecture defines every security layer protecting the BiteFixes Enterprise Platform.

Its objectives are to ensure:

- Confidentiality
- Integrity
- Availability
- Accountability
- Traceability
- Privacy
- Regulatory compliance

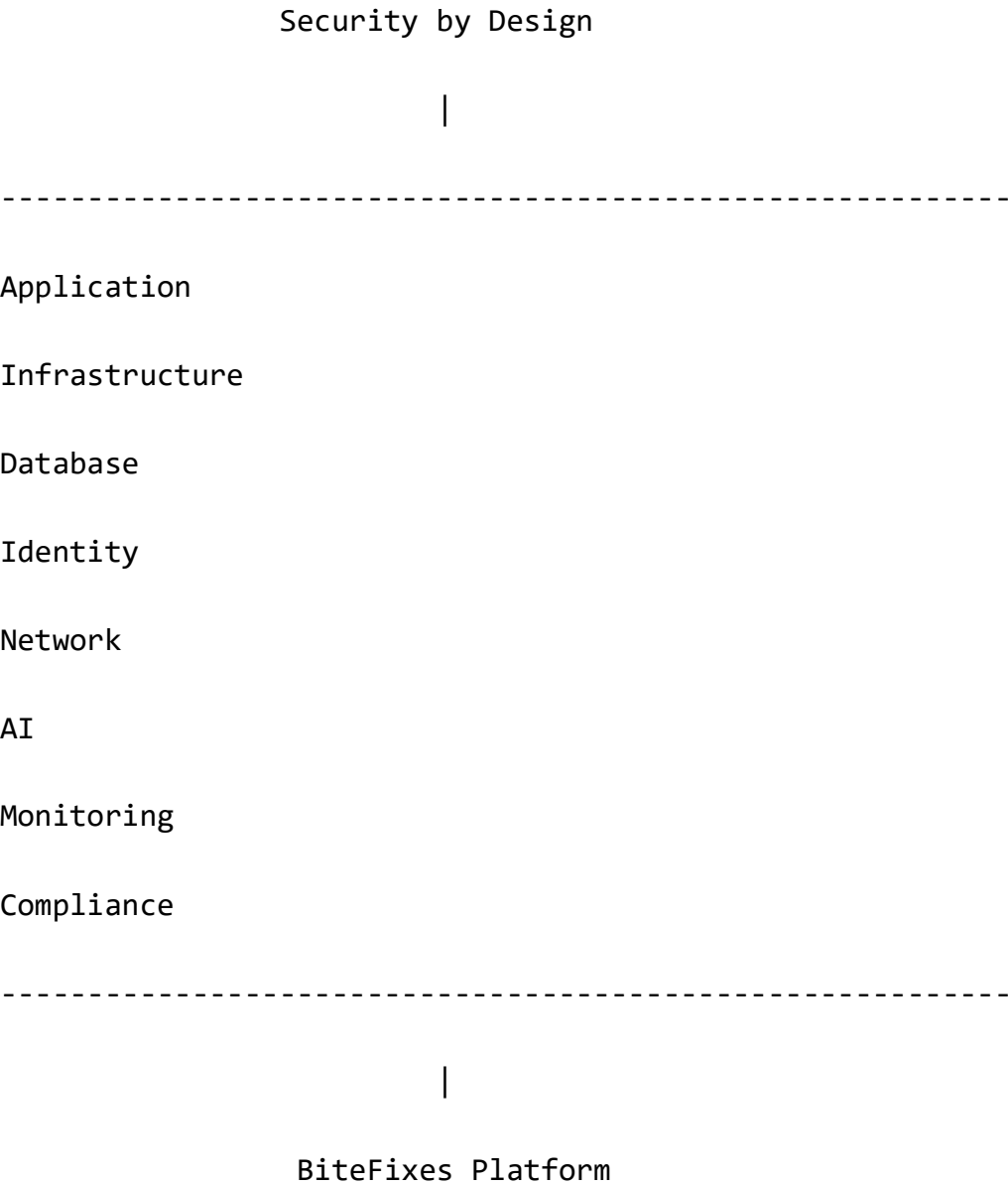
The architecture applies to every platform component:

- Bitey AI
- FastAPI Backend
- Supabase Platform
- Website
- Mobile Application
- WhatsApp Integration
- Future APIs
- Future Microservices

13.2 Security Vision

Security is not treated as an isolated feature.

Instead, security is embedded into every architectural layer.



Every request, service and database operation passes through one or more security controls before execution.

13.3 Security Principles

Principle 1 — Zero Trust

No request is trusted automatically.

Every operation requires verification.

Verification includes:

- Identity
- Authentication
- Authorization
- Context
- Company ownership
- Resource ownership

Principle 2 — Least Privilege

Every user receives only the permissions required for their role.

Example:

Customer

↓

Own Tickets

↓

Own Devices

↓

Own Conversations

Technicians cannot access financial data.

Customers cannot access internal diagnostics.

Companies cannot access another company's information.

Principle 3 — Defense in Depth

Multiple independent security layers protect every operation.

Internet



HTTPS



Cloud Protection



Authentication



Authorization



Business Rules



Database Security



Audit

Failure of one layer must not compromise the platform.

Principle 4 — Secure by Default

Every new feature starts from a secure baseline.

Examples:

- APIs require authentication unless explicitly public.
- Sensitive data is encrypted.
- Internal endpoints are hidden by default.
- Logging excludes secrets and credentials.
- Permissions are denied unless explicitly granted.

Principle 5 — Continuous Verification

Security is continuously evaluated during runtime.

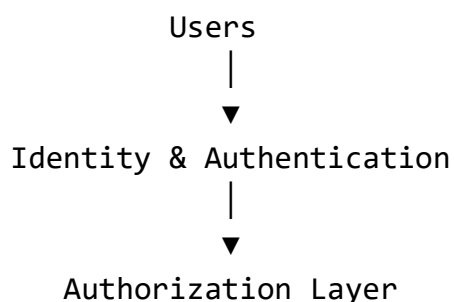
Checks include:

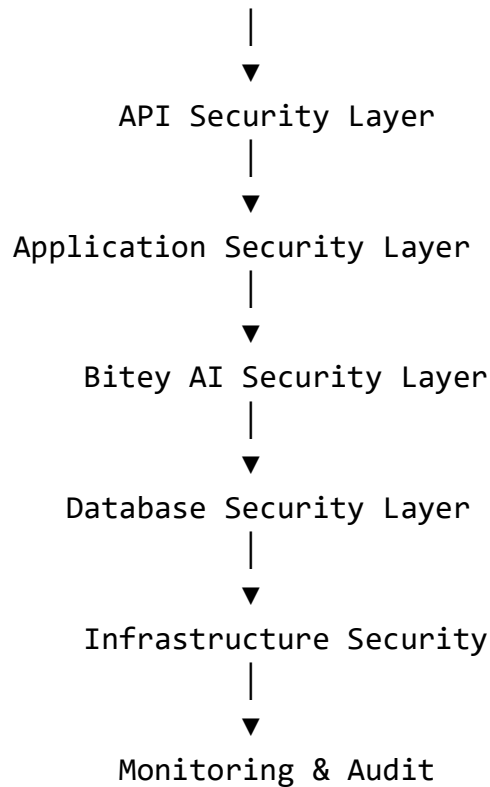
- Session validity
- Token expiration
- Permission changes
- Company ownership
- Resource access
- Suspicious activity

No long-lived trust assumptions are made.

13.4 Enterprise Security Layers

The platform is protected through multiple coordinated layers.





Each layer operates independently while sharing security events with the monitoring system.

13.5 Identity & Access Management (IAM)

Identity management is centralized to provide consistent authentication and authorization across all channels.

Supported identities include:

- Customers
- Technicians
- Company Administrators
- Platform Administrators
- API Clients
- Future AI Agents

Each identity is associated with:

- Unique identifier
- Authentication credentials

- Assigned roles
- Effective permissions
- Company association
- Session metadata
- Audit history

The IAM model is designed to support future Single Sign-On (SSO), Multi-Factor Authentication (MFA), and enterprise identity providers without requiring changes to the core domain model.

13.6 Authentication Architecture

Overview

Authentication is responsible for verifying the identity of every entity interacting with the BiteFixes Platform before any business operation is executed.

Authentication is independent from authorization.

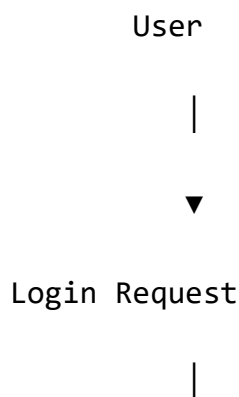
Authentication answers:

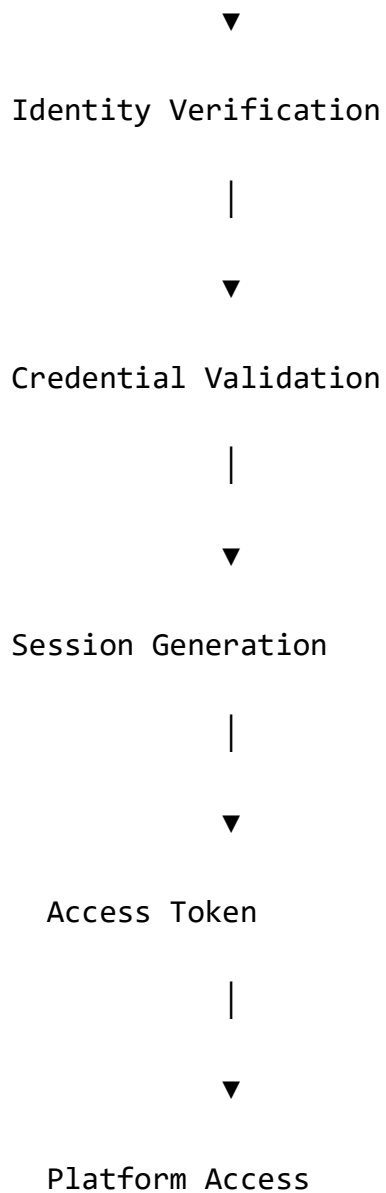
Who are you?

Authorization answers:

What are you allowed to do?

Authentication Flow





Authentication Sources

The platform supports multiple authentication providers.

Customer Login

Administrator Login

Technician Login

API Clients

Mobile Application

WhatsApp Identity

Future Enterprise SSO

Every authentication method eventually produces the same internal security context.

Authentication Context

After successful authentication, the runtime creates a Security Context.

Example:

SecurityContext

UserID

CompanyID

Role

Permissions

SessionID

AuthenticationMethod

ExpirationTime

RiskScore

This context accompanies every request until completion.

Session Lifecycle

Login

↓

Authentication

↓

Session Created

↓

Access Token

↓

Request Processing

↓

Token Renewal

↓

Logout / Expiration

↓

Session Destroyed

Expired sessions immediately lose access to protected resources.

13.7 Authorization Architecture

Overview

Authorization determines whether an authenticated identity may execute a requested operation.

Authorization is evaluated for every request.

Authorization Pipeline

Authenticated User

|



Resource Requested

|



Permission Evaluation

|



Company Validation

|



Ownership Validation

|



Business Rules

|



Access Granted / Denied

Role-Based Access Control (RBAC)

The first authorization layer is based on roles.

Example roles:

Customer

Technician

Service Manager

Company Administrator

Platform Administrator

API Integration

AI Agent (Future)

Each role aggregates one or more permissions.

Permission Model

Permissions follow an action/resource pattern.

Examples:

ticket.read

ticket.create

ticket.update

ticket.close

device.read

`device.update`

`customer.read`

`customer.update`

`company.manage`

`knowledge.manage`

`service.manage`

`user.manage`

`system.admin`

This naming convention provides consistency across backend services and APIs.

Fine-Grained Authorization

Beyond roles, additional rules are evaluated.

Examples:

A technician may update only tickets assigned to them.

A customer may only view their own devices.

A company administrator may manage only resources belonging to their company.

A platform administrator may access all tenants.

Authorization Decision

Every request results in one of two outcomes.

ALLOW

or

DENY

Denied requests are recorded in the audit log for security monitoring.

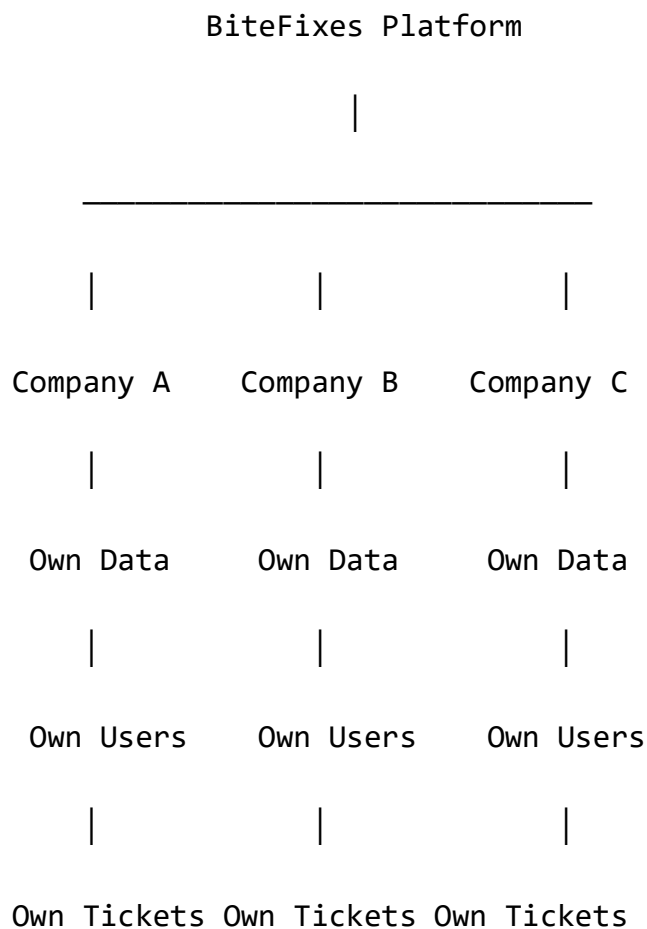
13.8 Multi-Tenant Security

Objective

BiteFixes is designed as a true multi-tenant SaaS platform.

Each company operates in complete isolation while sharing the same infrastructure.

Tenant Isolation Model



No tenant can access another tenant's information.

Tenant Resolution

Every authenticated request includes a Company Context.

Request

↓

Authentication

↓

Company Resolution

↓

Tenant Validation

↓

Business Services

The Company Context is mandatory before accessing any business resource.

Row-Level Isolation

Every business entity includes a tenant identifier.

Example:

CompanyID

CustomerID

DeviceID

TicketID

ConversationID

KnowledgeID

InvoiceID

Queries are always filtered by CompanyID.

Example:

```
WHERE company_id = current_company
```

This rule applies consistently across all services and APIs.

Cross-Tenant Protection

The platform explicitly prevents:

- Reading another company's data.
- Updating another company's records.
- Executing reports across unauthorized tenants.
- Sharing AI memory between companies.
- Accessing foreign ticket histories.
- Viewing another company's knowledge base.

Tenant isolation is enforced at the application, database, and authorization layers.

13.9 API Security

Overview

Every API endpoint is protected according to its exposure level.

The platform distinguishes between:

Public APIs

Protected APIs

Internal APIs

Administrative APIs

Partner APIs

Each category has different authentication and authorization requirements.

API Security Pipeline

Incoming Request

|



HTTPS Validation

|



Rate Limiting

|

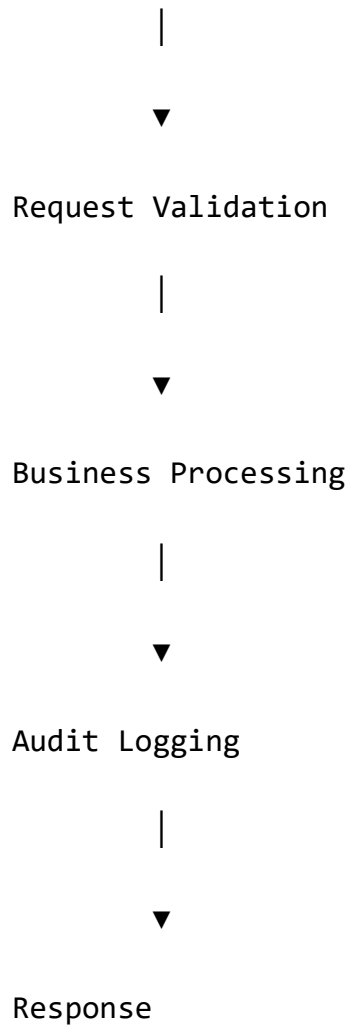


Authentication

|



Authorization



Every request passes through the complete security pipeline before reaching the business logic.

Request Validation

All incoming requests are validated for:

- JSON structure
- Required fields
- Data types
- Value ranges
- Payload size
- Content encoding
- Input sanitization

Malformed or suspicious requests are rejected before business processing.

API Rate Limiting

To protect platform availability, request limits are enforced.

Example policies:

- Anonymous requests: highly restricted.
- Authenticated customers: standard limits.
- Technicians: elevated limits.
- Administrators: managed limits.
- Internal services: trusted channels with monitoring.

Rate limiting mitigates abuse, brute-force attempts, and denial-of-service scenarios.

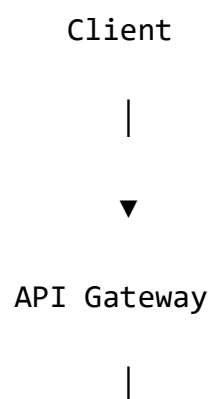
13.10 Application Security

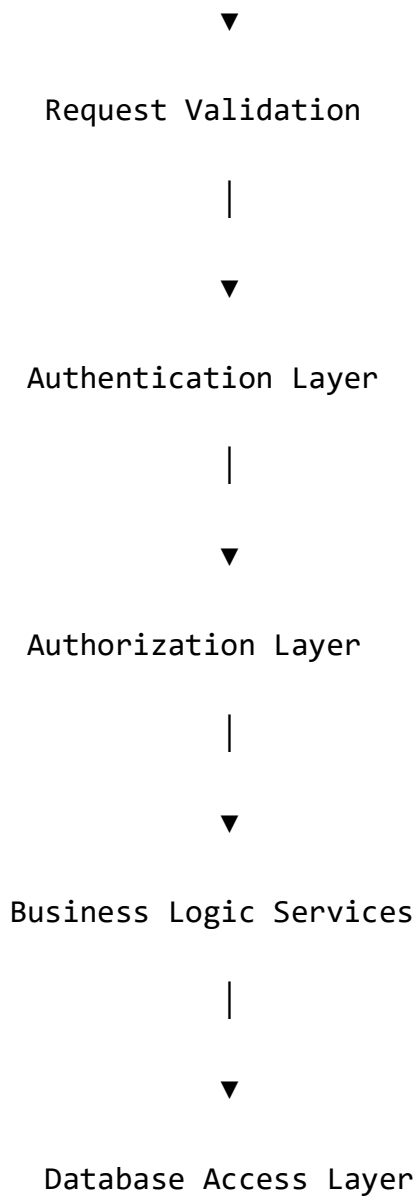
Overview

Application Security protects the business logic of the BiteFixes Platform against unauthorized access, malicious input, software vulnerabilities, and runtime attacks.

Unlike infrastructure security, application security focuses on the behavior of the software itself.

Secure Application Architecture





Each layer validates the request before passing control to the next component.

Secure Coding Standards

All backend services must follow secure development practices.

Input Validation

Every external input is validated using strict schemas.

Validation includes:

- Required fields
- Data types
- Length restrictions
- Character restrictions
- Enum validation
- Date validation
- UUID validation

Output Validation

Responses must never expose:

- Internal exceptions
- Stack traces
- Database schema
- SQL statements
- Secrets
- API keys
- Internal configuration

Exception Handling

Unexpected exceptions are:

- Logged internally
- Assigned a correlation identifier
- Returned as generic error messages

Example:

Request

↓

Unexpected Exception

↓

Internal Log

↓

Generic Error Response

Dependency Security

Every dependency used by the platform should be:

- Version controlled
- Regularly updated
- Security scanned
- Reviewed before adoption

Third-party libraries must follow approved security policies.

13.11 Database Security

Objectives

Protect:

- Customer information
- Business information
- AI memory
- Tickets
- Conversations
- Knowledge base
- Internal configuration

Database Security Layers

Application

|

▼

ORM / Repository

|



Row-Level Security

|



Database Roles

|



Encrypted Storage

Row-Level Security (RLS)

Supabase Row-Level Security is mandatory for all business tables.

Example policy:

Authenticated User

|



CompanyID Match

|



Record Accessible

No query may bypass tenant isolation.

Database Roles

The database uses multiple logical roles.

Anonymous

Authenticated User

Technician

Company Administrator

Platform Administrator

Internal Services

Each role has independent permissions.

Encryption

Sensitive information is protected through encryption.

Examples include:

- Authentication credentials
- API secrets
- Password hashes
- Personal information
- Session tokens

Encryption is applied:

- At rest
- In transit
- During authentication workflows

Backup Protection

Database backups are:

- Encrypted
- Versioned
- Integrity verified
- Access restricted

Only authorized administrators may initiate restoration procedures.

13.12 AI Security (Bitey Security Model)

Overview

Bitey is a core business component and therefore requires dedicated security controls beyond traditional application security.

The AI engine must operate safely, predictably, and within strict organizational boundaries.

Bitey Security Pipeline

User Message

|



Input Validation

|



Intent Detection

|



Security Filters

|



Context Builder

|



Knowledge Retrieval

|



Response Generation

|



Output Validation

|



Customer Response

Security checks occur both before and after AI processing.

Prompt Injection Protection

The platform validates incoming prompts to detect attempts to manipulate system behavior.

Examples include attempts to:

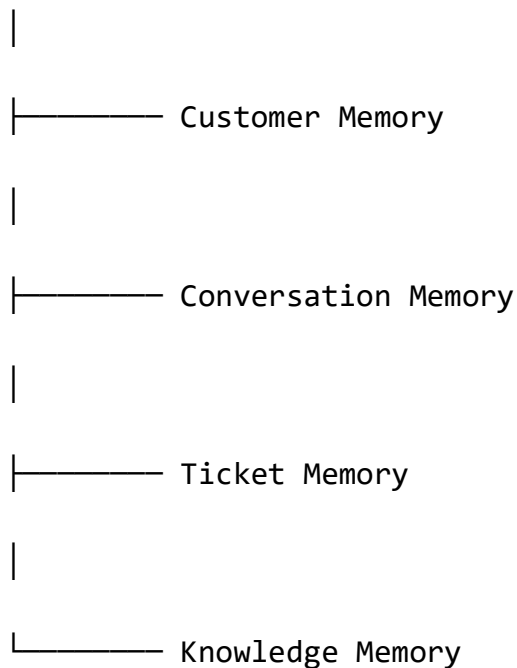
- Reveal internal prompts
- Ignore platform instructions
- Access hidden knowledge
- Execute unauthorized actions
- Escalate privileges
- Bypass tenant isolation

Suspicious prompts are blocked or sanitized before processing.

Memory Isolation

Bitey maintains separate memory contexts.

Company Memory



Memory belonging to one company is never accessible from another tenant.

AI Decision Validation

Before executing any business action, Bitey validates:

- Confidence score
- Intent classification
- User permissions
- Company ownership
- Business rules

Only validated decisions may trigger actions such as ticket creation or service updates.

AI Auditability

Every AI interaction records:

- Timestamp
- User
- Company
- Detected intent
- Confidence score
- Knowledge source
- Response identifier
- Executed actions

This enables full traceability and continuous model evaluation.

13.13 Data Privacy

Privacy Objectives

BiteFixes is designed to minimize exposure of personal and business information while ensuring operational efficiency.

Privacy principles apply to:

- Customers
- Companies
- Employees
- Devices
- Conversations
- AI-generated data

Data Classification

Information is classified into four levels.

Public

↓

Internal

↓

Confidential

↓

Restricted

Each classification determines:

- Storage requirements
- Encryption level
- Access permissions
- Retention period
- Audit requirements

Personal Data

Examples include:

- Customer names
- Email addresses
- Phone numbers
- Device identifiers
- Service history

These data elements are processed only for legitimate business purposes and protected throughout their lifecycle.

Data Retention

Information is retained according to business and legal requirements.

Operational Data

↓

Archive

↓

Retention Review

↓

Secure Deletion

Deletion processes ensure that obsolete data cannot be recovered through normal platform operations.

Privacy by Design

Privacy requirements are incorporated during system design rather than added later.

Every new feature must define:

- Data collected
- Purpose of collection

- Access rules
- Retention policy
- Deletion strategy
- Audit requirements

This ensures that privacy remains an architectural principle across the entire BiteFixes platform.

13.14 Secrets Management

Overview

Secrets Management ensures that all sensitive credentials used by the BiteFixes Platform are securely stored, rotated, and accessed only by authorized runtime components.

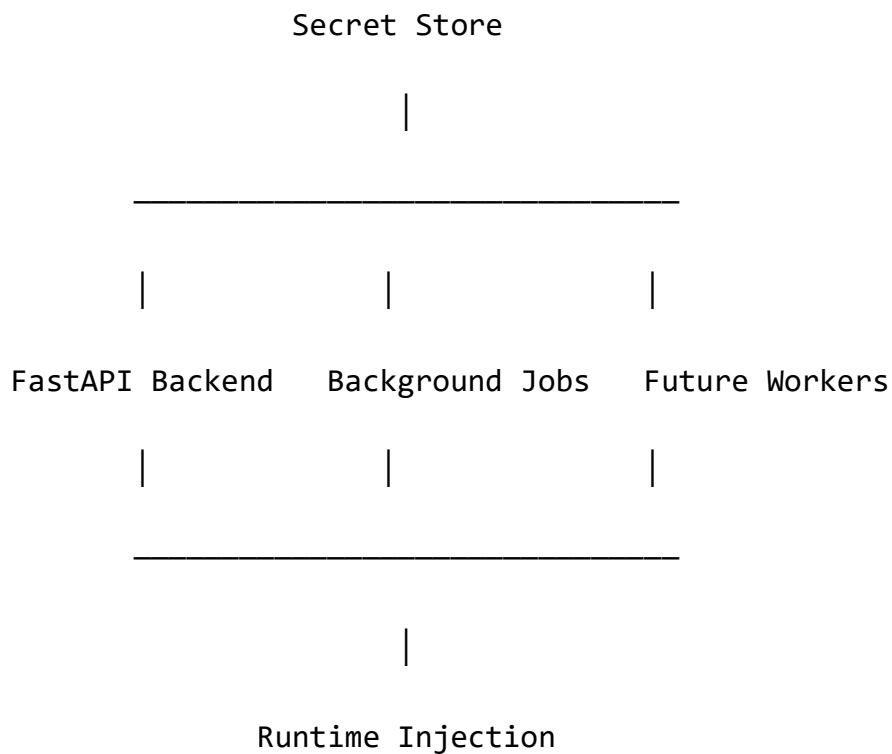
Secrets are never hardcoded into the application source code.

Managed Secrets

The platform manages secrets such as:

- Database credentials
- JWT signing keys
- Encryption keys
- WhatsApp API tokens
- OpenAI API keys
- Google Gemini API keys
- SMTP credentials
- Payment gateway credentials
- Push notification keys
- Third-party integration tokens

Secrets Architecture



Secrets are loaded only during runtime and are never exposed to clients or logs.

Secret Rotation

Every secret should support periodic rotation without requiring application redesign.

Rotation strategy includes:

- Scheduled rotation
- Emergency rotation
- Automatic invalidation
- Audit recording
- Rollback capability

Environment Separation

Secrets are isolated by environment.

Development

Testing

Staging

Production

Production secrets must never be reused in lower environments.

13.15 Audit & Logging

Objectives

Every relevant platform action must be traceable.

Audit logging supports:

- Security investigations
- Compliance
- Operational monitoring
- AI analysis
- Incident response

Audit Architecture

Application Event

|



Audit Service

|



Immutable Audit Log



Monitoring Dashboard



Security Analysis

Auditable Events

The following actions are recorded:

Authentication

- Login
- Logout
- Failed login
- Password reset
- Token refresh

User Management

- User creation
- User deletion
- Role assignment
- Permission changes
- Account suspension

Customer Operations

- Customer creation

- Customer update
- Customer deletion

Device Operations

- Device registration
- Diagnostic creation
- Repair updates

Ticket Operations

- Ticket creation
- Assignment
- Status changes
- Closure
- Reopening

AI Operations

- Intent detected
- Knowledge source selected
- Confidence score
- Automatic ticket creation
- Generated recommendation

Administrative Actions

- Configuration changes
- Company creation
- Service catalog modifications
- Security policy updates

Log Integrity

Audit records are immutable.

An audit event cannot be modified after being stored.

Each record includes:

- Timestamp
- Actor
- Company
- Resource
- Action
- Previous state
- New state
- Correlation ID
- IP Address
- User Agent

13.16 Threat Detection

Security Monitoring

The platform continuously evaluates abnormal behavior.

Threat analysis combines:

- Runtime monitoring
- Behavioral analysis
- Authentication events
- API activity
- AI interactions

Threat Detection Pipeline

Incoming Activity

|



Security Rules

|

▼

Behavior Analysis

|

▼

Risk Scoring

|

▼

Threat Classification

|

▼

Alert Generation

|

▼

Security Dashboard

Detected Threat Categories

Identity Threats

- Credential stuffing
- Brute force
- Account takeover
- Session hijacking

API Threats

- Excessive requests
- Malformed payloads
- Injection attempts
- Invalid authentication

AI Threats

- Prompt injection
- Context manipulation
- Unauthorized memory access
- Jailbreak attempts
- Sensitive data extraction

Infrastructure Threats

- Denial of Service
- Network anomalies
- Resource exhaustion
- Unauthorized deployments

Risk Levels

Low

Medium

High

Critical

Risk levels determine automated response actions.

13.17 Backup & Disaster Recovery

Objectives

Guarantee business continuity in the event of:

- Infrastructure failures
- Database corruption
- Human error
- Cyber attacks
- Cloud provider incidents

Backup Strategy

Production

|



Continuous Backup

|



Encrypted Storage

|



Recovery Validation

|



Disaster Recovery

Recovery Objectives

Recovery Point Objective (RPO)

Maximum acceptable data loss between backups.

Recovery Time Objective (RTO)

Maximum acceptable downtime before restoring service.

Target values will be defined according to the selected service tier (Standard, Professional, Enterprise).

Disaster Recovery Process

Incident

↓

Detection

↓

Assessment

↓

Recovery Decision

↓

Backup Restoration

↓

Validation

↓

Production Recovery

↓

Post-Incident Review

13.18 External Integrations Security

The platform integrates securely with external providers.

Current integrations include:

- WhatsApp Cloud API
- WordPress
- OpenAI
- Google Gemini
- Email providers
- Push notification services

Future integrations may include:

- Payment gateways
- ERP systems
- CRM platforms
- Marketplace APIs
- Accounting software

Integration Security Controls

Each integration must implement:

- HTTPS communication
- API authentication
- Secret management
- Retry policies
- Timeout protection

- Request validation
- Response validation
- Rate limiting
- Audit logging

Webhook Security

Incoming webhooks must include:

- Signature verification
- Timestamp validation
- Replay protection
- Source validation

Unverified webhooks are rejected.

13.19 Compliance Strategy

Security Governance

The architecture is designed to facilitate compliance with internationally recognized security and privacy standards.

Target frameworks include:

- ISO/IEC 27001
- ISO/IEC 27701
- SOC 2 (Type II)
- NIST Cybersecurity Framework
- OWASP ASVS
- OWASP Top 10

Privacy Regulations

The platform is designed to support compliance with regulations such as:

- LGPD (Brazil)
- GDPR (European Union)
- CCPA (California), where applicable

Compliance requirements are implemented through configurable policies rather than hardcoded regional logic.

Secure Development Lifecycle

Every feature follows a secure development lifecycle:

Requirements

↓

Architecture Review

↓

Implementation

↓

Security Testing

↓

Code Review

↓

Deployment

↓

Monitoring

↓

Continuous Improvement

13.20 Security Roadmap

Evolution Strategy

Security capabilities evolve alongside platform maturity.

Phase 1 — Foundation

- Authentication
- Authorization
- Multi-tenant isolation
- API security
- Audit logging

Phase 2 — Enterprise Security

- Multi-Factor Authentication (MFA)
- Single Sign-On (SSO)
- Advanced RBAC
- Conditional Access
- Centralized Secrets Management

Phase 3 — Intelligent Security

- AI-assisted threat detection
- Behavioral anomaly detection
- Automated incident response
- Adaptive authentication
- Risk-based authorization

Phase 4 — Global Enterprise Platform

- Security Operations Center (SOC) integration
- Compliance automation

- Continuous control monitoring
- Zero Trust across distributed services
- Cross-region resilience

Volume Summary

The **Platform Security Architecture (PSA)** establishes the security foundation of the BiteFixes Enterprise Platform by integrating protection mechanisms across identity, application, data, AI, infrastructure, and external integrations.

Rather than treating security as an isolated subsystem, the architecture embeds security controls throughout every layer of the platform. This approach enables secure multi-tenant operation, protects sensitive customer and company data, safeguards the Bitey AI engine, and supports future compliance with enterprise security standards while maintaining scalability and operational resilience.